

FORM-IC-01

Investigación Individual

Arquitecturas Distribuidas en la Industria

Alumna: Juliana Emanuel Pino
Carrera: 7to Software

Descripción del Caso: Netflix y su Migración a Microservicios

Netflix es una de las plataformas de *streaming* más grandes del mundo, con más de 260 millones de suscriptores activos en más de 190 países [2]. Hasta 2008, operaba con una arquitectura monolítica hospedada en centros de datos propios. Una falla crítica en su base de datos relacional ese año dejó el servicio inoperante por varios días, lo que motivó a la empresa a repensar completamente su infraestructura.

Entre 2009 y 2016, Netflix llevó a cabo una migración total hacia una arquitectura basada en microservicios sobre la nube de Amazon Web Services (AWS). Cada capacidad del sistema —catálogo de contenido, autenticación, recomendaciones, facturación, gestión de perfiles, codificación de video— fue desacoplada en un servicio independiente con su propia base de datos, ciclo de despliegue y equipo dedicado [1].

¿Por Qué Eligieron la Arquitectura de Microservicios?

Los principales factores que impulsaron la decisión fueron:

- **Escalabilidad granular:** el tráfico de Netflix tiene picos muy pronunciados (es-trenos nocturnos, fines de semana). Con microservicios, solo se escalan los servicios bajo demanda (ej. codificación de video) sin escalar toda la aplicación.
- **Independencia de despliegue:** los equipos pueden lanzar nuevas versiones de un servicio sin coordinar con el resto, alcanzando cientos de despliegues diarios.
- **Resiliencia mediante aislamiento de fallos:** si el servicio de recomendaciones falla, el usuario puede seguir reproduciendo contenido, pues el motor de *streaming* es un servicio separado.
- **Autonomía tecnológica:** cada equipo puede elegir el lenguaje o motor de base de datos más adecuado para su dominio.

Beneficios Obtenidos

La transición le reportó a Netflix mejoras medibles y sostenibles:

- **Alta disponibilidad:** Netflix alcanzó un *uptime* superior al 99,99%, eliminando las ventanas de mantenimiento que paralizaban el sistema monolítico.

- **Velocidad de entrega:** redujo el ciclo de despliegue de semanas a minutos, habilitando integración y entrega continuas (CI/CD).
- **Innovación acelerada:** la independencia de servicios permitió experimentar con algoritmos de recomendación sin riesgo para el núcleo del sistema.
- **Reducción de costos operativos:** al escalar solo los componentes necesarios, se optimizó significativamente el uso de infraestructura en la nube [2].

Desafíos Encontrados

La migración también implicó retos importantes que Netflix debió resolver:

- **Complejidad operacional:** gestionar cientos de servicios independientes requirió crear herramientas propias como Eureka (registro de servicios) e Hystrix (circuit breaker), que luego liberó como código abierto.
- **Consistencia de datos distribuidos:** garantizar la coherencia entre servicios que tienen bases de datos propias exigió adoptar el modelo de consistencia eventual y patrones como Saga.
- **Latencia en comunicaciones:** las llamadas entre servicios introducen latencia de red. Netflix debió implementar cachés distribuidas (EVCache) y optimizar sus APIs internas.
- **Observabilidad:** depurar un error que atraviesa veinte servicios es mucho más difícil que en un monolito; requirió invertir en trazas distribuidas y dashboards unificados.

Aplicación al Proyecto: SGA-Soporte (UTEQ)

El proyecto *sga-soporte* desarrollado para el Sistema de Gestión Académica de la Universidad Técnica de Machala sigue directamente el patrón de microservicios que Netflix adoptó. Se trata de un microservicio independiente implementado en Spring Boot (puerto 8081) que expone una API REST para la gestión de tickets de soporte técnico, desacoplado del módulo principal sga-principal.

Al igual que Netflix separa su motor de streaming de sus recomendaciones, *sga-soporte* separa la lógica de soporte —tickets, comentarios, estadísticas, notificaciones por correo— del núcleo académico (matrículas, calificaciones). Ambos módulos comparten la misma base de datos PostgreSQL pero operan en esquemas independientes (**soporte** vs. **public**), y se comunican mediante tokens JWT compartidos, replicando el patrón de autenticación distribuida utilizado por Netflix con su servicio de identidad.

Los desafíos identificados en Netflix también son aplicables al proyecto: si el microservicio de soporte queda inactivo, el flujo académico principal no se ve interrumpido, lo que valida la resiliencia por aislamiento. Como mejora futura, se podría incorporar un *circuit breaker* con Resilience4j y un API Gateway (Spring Cloud Gateway) que centralice el enrutamiento, tal como lo hace Netflix con su capa de Zuul/Envoy.

Referencias

- [1] Lewis, J. y Fowler, M. (2014). *Microservices: a definition of this new architectural term*. martinowler.com. Recuperado de <https://martinfowler.com/articles/microservices.html>
- [2] Mauro, T. (2015). *Adopting Microservices at Netflix: Lessons for Architectural Design*. NGINX Blog. Recuperado de <https://www.nginx.com/blog/microservices-at-netflix-architectural-best-practices/>